

Examen réparti

Carlos Agon

Déroulement de L'examen

Tous les documents sont autorisés, et notamment ceux du cours.

L'utilisation de tout moyen de communication (téléphone, tablette, ordinateur personnel, etc.) est interdite.

L'utilisation d'une clé ou disque USB personnel, en lecture seule et étiqueté à votre nom, est autorisée; vous ne devez pas le prêter à un camarade.

Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction. **L'examen dure 2h.** Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `eclipse-workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

Le langage à étendre est ILP2. Le packaging Java correspondant sera nommé `com.paracamplus.ilp2.ilp2partiel`. Sera ramassé, à partir de votre `~/eclipse-workspace` (situé sous ce nom directement dans votre HOME), le **seul** répertoire `~/eclipse-workspace/ILP2/Java/src/com/paracamplus/ilp2/ilp2partiel/` et tous ses sous-répertoires.

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve.

Mise en route

- Lancez Eclipse avec `eclipse-2020 &`
- Choisissez le repertoire `eclipse-workspace` comme workspace
- Si le projet ILP2 n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `eclipse-workspace/ILP2`, puis « Finish ».
- En cas de problème de compilation, il est possible que l'environnement Java JRE soit mal configuré. Faites un clic droit sur le nom du projet dans la fenêtre « Package Explorer », cliquez sur « Properties » puis, dans l'onglet « Java build path / Libraries » vérifiez la version de la bibliothèque système utilisée (« JRE System Library »). Vous pouvez changer de bibliothèque avec le bouton « Edit... / Alternate JRE » et choisir un JDK approprié (JDK 11 ou supérieur). Les JDK disponibles localement sont dans le répertoire `/usr/lib/jvm`.
- le plugin ANTLR devrait fonctionner sans réglages particuliers. Toutefois, si ce n'est pas le cas, vous pouvez toujours utiliser ANTLR en ligne de commande à l'aide d'un script ; il suffit de faire :

```
cd ~/eclipse-workspace/ILP2
./compile_ANTLR.sh
```

On rappelle que l'on peut aussi lancer le plug-in à la main depuis Eclipse avec "Run as..." si il ne se lance pas automatiquement lors de l'édition d'une grammaire.

Dépannage

- Si jamais eclipse se bloque en essayant de se connecter à svn, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez eclipse.

- Ouvrez un navigateur et allez à l'URL :

<http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>

Vous y trouverez notamment les transparents du cours.

Introduction

Dans cet examen, nous voulons étendre ILP2 avec une nouvelle expression appelée `sequenceN`. L'évaluation d'une expression de type `sequence` en ILP évalue dans l'ordre toutes les expressions de la `sequence` et donne comme résultat la valeur de l'évaluation de la dernière expression. Par exemple l'évaluation du programme suivant imprime "9".

```
x=1
y=(1+1; 2+2; 3+3; 4+4)
print(x+y)
```

Il est utile, parfois, d'évaluer plusieurs expressions dans une `sequence` et de renvoyer le résultat de la première ou de la deuxième plutôt que de la dernière expression. C'est pour cela que nous voulons introduire dans ILP2 la nouvelle expression `sequenceN`. Voici un exemple qui illustre cette nouvelle expression :

```
x=1
y={1*0 : 2+2 ; 3+3 ; 4+4}
print(x+y)
```

A différence d'une `sequence` une `sequenceN` contient une liste d'expressions entre accolades (i.e. '{ }'). L'évaluation de la première expression donne un entier qui spécifie l'index de l'expression dont sa valeur sera retournée. Cette première expression est séparée des expressions suivantes par le symbole ' : '. Les expressions suivantes sont évaluées l'une après les autres comme dans une `sequence` normale. Ainsi dans notre exemple précédant la programme imprimera "5" car pour la première expression le résultat est 0, donc notre `sequenceN` évaluera les trois autres expressions et retournera la valeur de la première expression après les ' : ', à savoir 4.

Et voici d'autres exemples :

```
x=0
y={x+1 : 2; 3;4}
print(x+y)
```

Le programme imprime "4" car `x+1` vaut 1 et `y` vaut 3.

```
x=1
y={{x-1 : 1} : 2; 3; 4}
print(x+y)
```

Le programme imprime "4"

1 Grammaire (2 points)

Ecrivez une grammaire dans le fichier :

```
com/paracampus/ilp2/ilp2partiel/grammars/ILPMLgrammarPartiel.g4
```

en incluant la nouvelle expression `sequenceN`.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique pour gérer l'introduction de `sequenceN`. On rappelle que vous devez fournir les classes AST ainsi que les interfaces correspondantes dans `com.paracampus.ilp2.ilp2partiel.ast` et `com.paracampus.ilp2.ilp2partiel.interfaces`.

On n'oubliera pas d'inclure la classe `ASTfactory` correspondante et son interface `IASTfactory` ainsi que la nouvelle interface `IASTvisitor`.

3 Analyse syntaxique (2 points)

Ecrivez dans le package `com.paracamplus.ilp2.ilp2partiel.parser` les nouvelles classes `ILPMLParser` et `ILPMLListener` permettant de construire les nouveaux nœuds ajoutés au langage.

4 Interprétation (5 points)

Implantez un nouvel interpréteur dans une classe `com.paracamplus.ilp2.ilp2partiel.interpreter.Interpreter`.

5 Compilation (6 points)

5.1 Questions (2 points)

Répondez aux questions suivantes :

- listez les classes et interfaces que vous auriez à ajouter pour mettre en œuvre le compilateur, en précisant pour chacune les classes et interfaces qu'elles étendent ou implantent ;
- listez les différents erreurs à détecter lors de l'exécution d'une `sequenceN`.

5.2 Schéma de compilation (4 points)

Proposez les schémas de compilation pour l'expression `sequenceN` dans un fichier `Compiler.txt` dans le package `com.paracamplus.ilp2.ilp2examen.compiler`. Quelles erreurs pouvez-vous détecter statiquement et quelles erreurs ne peuvent être détectées que dynamiquement ? Si vous croyez qu'il y a des erreurs dynamiques qu'impliquent des changements à la bibliothèque d'exécution, ajoutez le code C nécessaire dans un fichier `ilp.c`.

6 Tests (2 points)

Créez un répertoire `com/paracamplus/ilp2/ilp2partiel/SamplesPartiel/` dans lequel vous mettrez un ensemble de programmes pour tester votre implantation. Pour chaque programme expliquez l'erreur ou le comportement que vous testez. Donnez le résultat et les effets de bord attendus.